

Audit du passage de C05 C06 vers la vraie IA HOME AI

Ce document décrit le parcours client actuel, l'utilisation du diagnostic local, l'infrastructure OpenAI existante et un plan V1 minimal pour appeler une IA via Supabase Edge Functions sans exposer OPENAI_API_KEY dans GitHub Pages.

1 Parcours actuel C05 C06 C07 et création de mission

- C05 : le client saisit une description du problème. Le formulaire appelle diagnostic.analyse avec la description et la catégorie éventuellement choisie.
- C06 : le résultat fournit actuellement categoryId et summary. L'interface affiche le problème compris, le service correspondant et le type de technicien, puis propose de chercher un prestataire.
- C07 : le clic sur la recherche récupère la position client, vérifie la session Supabase et charge les repositories.
- En mode Supabase, createCustomerMissionDraft construit la mission avec la catégorie, la description originale, le résumé du diagnostic, l'adresse et les coordonnées.
- createOrResume vérifie d'abord s'il existe déjà une mission récupérable. Si nécessaire, createCurrent appelle create_current_customer_mission, puis createOffers appelle create_current_customer_mission_offers.
- Realtime et le polling démarrent après la création ou la reprise de la mission. En mode Supabase, la mission est créée au lancement de la recherche C07. En mode mock, l'ancien parcours continue avec une sélection locale puis une confirmation.

2 Utilisation actuelle de createMockDiagnostic

createMockDiagnostic est importé dans src/app.js et utilisé comme dépendance par défaut de initialiseHomePage. Son implémentation se trouve dans src/diagnostic/mock-diagnostic.js. Elle classe le texte avec des mots-clés déterministes, renvoie categoryId et summary, et contient un résultat enrichi particulier pour le cas de la climatisation qui ne refroidit pas. En production GitHub Pages, aucune autre implémentation n'est injectée : C05 utilise donc toujours ce moteur local.

3 Infrastructure LLM et OpenAI déjà présente

- LLMService et une factory de providers mock ou OpenAI.
- Un client OpenAI Responses API avec délai maximal, AbortController et traitement des erreurs 401, 429, 5xx, réponse invalide et erreur réseau.
- Extraction de output_text et début de Structured Outputs JSON.
- Un orchestrateur hybride avec fallback déterministe.
- Des tests unitaires du provider OpenAI.

Cette infrastructure fonctionne dans un environnement Node disposant de variables serveur, mais elle n'est pas branchée sur C05 C06. src/api.js est un serveur Node absent du déploiement statique GitHub Pages. Le planner existant concerne un prototype home status lights et non le diagnostic HOME AI. Le schéma JSON OpenAI actuel est permissif et ne définit pas les propriétés obligatoires.

4 Éléments manquants pour un appel sécurisé

- Une Edge Function dédiée, par exemple diagnose-home-request.
- OPENAI_API_KEY configurée comme secret Supabase Edge Functions.
- Validation de la session client Supabase.
- Validation stricte du texte d'entrée et limitation de sa taille.
- Schéma de sortie strict et validation serveur du JSON retourné.

- Client frontend utilisant `supabase.functions.invoke`.
- Timeout, gestion des erreurs et fallback local sûr.
- Tests de contrat Edge Function et tests du parcours C05 C06.

OPENAI_API_KEY ne doit jamais être ajoutée à `runtime-config.js`, au build dist, aux secrets GitHub Pages injectés dans le navigateur ou à un module frontend.

5 Architecture avec Supabase Edge Function

1. Le client GitHub Pages invoque `diagnose-home-request` avec son jeton de session Supabase.
2. La fonction vérifie l'utilisateur, l'origine et le contenu de la requête.
3. La fonction lit `OPENAI_API_KEY` avec `Deno.env.get` et appelle OpenAI côté serveur.
4. La réponse OpenAI est validée et normalisée avant d'être retournée au navigateur.
5. La V1 n'a besoin d'aucune écriture en base, d'aucun rôle `service_role` et d'aucune modification RLS.
6. Les erreurs doivent utiliser des codes stables et ne jamais journaliser la clé ou des données personnelles inutiles.

État actuel du projet Supabase : aucune Edge Function n'est déployée.

6 JSON structuré minimal

```
{
  "serviceCategory": "electricity",
  "understoodProblem": "Le client signale une panne électrique dans la maison.",
  "confidence": 0.92,
  "missingQuestions": [],
  "vietnameseSummary": "Hệ thống điện trong nhà gặp sự cố và cần thợ điện kiểm tra."
}
```

- `serviceCategory` : electricity, plumbing, air-conditioning ou appliances.
- `understoodProblem` : chaîne courte et factuelle.
- `confidence` : nombre compris entre 0 et 1.
- `missingQuestions` : tableau de chaînes, limité idéalement à trois questions.
- `vietnameseSummary` : résumé vietnamien affichable et persistable.

Un adaptateur frontend peut mapper `serviceCategory` vers `categoryId` et `vietnameseSummary` vers `summary` afin de conserver les consommateurs actuels.

7 Fallback sûr

Le fallback doit réutiliser `createMockDiagnostic` lorsque la fonction dépasse son délai, que le réseau est indisponible, qu'OpenAI répond 429 ou 5xx, ou que la réponse JSON est invalide. Il doit utiliser uniquement le texte fourni et les règles locales existantes, sans inventer d'urgence, de prix ou de diagnostic technique détaillé.

Le résultat doit être marqué comme fallback en mémoire. Une erreur d'authentification Supabase ne doit jamais être transformée en session mock. Les questions manquantes peuvent être affichées sans bloquer la recherche dans la V1.

8 Changements nécessaires

Frontend

- Créer `createSupabaseAiDiagnostic`.
- Appeler `supabase.functions.invoke` depuis C05.
- Adapter le résultat au contrat `categoryId` et `summary` existant.

- Afficher confiance et questions manquantes dans C06 si disponibles.
- Conserver le diagnostic local comme fallback contrôlé.

Backend Edge Function

- Créer diagnose-home-request.
- Valider l'utilisateur, l'origine, la méthode, le type de contenu et la longueur du texte.
- Appeler OpenAI avec un schéma JSON strict.
- Valider et normaliser la réponse avant retour.
- Normaliser les erreurs sans exposer les secrets.

Supabase

- Ajouter OPENAI_API_KEY aux secrets Edge Functions.
- Déployer la fonction authentifiée.
- Aucune migration ou modification RLS nécessaire pour la V1 minimale.

Plan d implémentation V1 minimal

1. Créer un contrat partagé de diagnostic avec validation stricte.
2. Créer l'Edge Function diagnose-home-request.
3. Protéger la fonction avec la session Supabase client.
4. Stocker OPENAI_API_KEY uniquement dans les secrets Edge Functions.
5. Appeler OpenAI avec un schéma strict et les quatre catégories autorisées.
6. Ajouter createSupabaseAiDiagnostic côté frontend.
7. Choisir ce diagnostic lorsque la session Supabase est réelle.
8. Conserver createMockDiagnostic pour le développement local et le fallback technique.
9. Adapter C06 pour afficher résumé, catégorie, confiance et questions éventuelles.
10. Garder createCustomerMissionDraft, le matching et le workflow mission inchangés.
11. Tester succès, JSON invalide, timeout, 429, 5xx, session absente et fallback.
12. Déployer la fonction et vérifier ses logs sans exposer le texte client ou la clé.